

AI Lab Experiments (1–20)

Experiment 1

Aim:

To install the Matplotlib library and create a simple line graph for data visualization.

Algorithm:

1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

```
plt.plot(x, y)
```

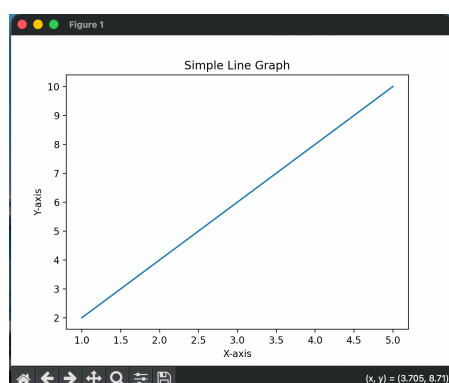
```
plt.title("Simple Line Graph")
```

```
plt.xlabel("X-axis")
```

```
plt.ylabel("Y-axis")
```

```
plt.show()
```

Output:



Result:

The experiment was executed successfully and the expected output was obtained.

Experiment 2

Aim:

To install the Scikit-learn library and verify its installation by displaying its version.

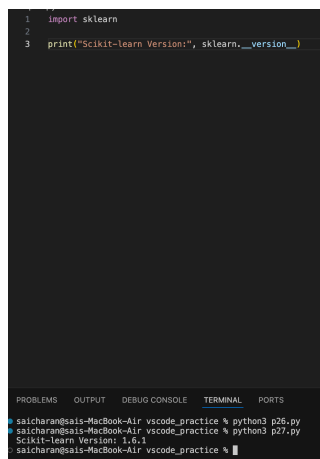
Algorithm:

1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
import sklearn  
  
print("Scikit-learn Version:", sklearn.__version__)
```

Output:

A screenshot of a code editor with a dark background. The editor shows three lines of Python code: `1 import sklearn`, `2` (blank line), and `3 print("Scikit-learn Version:", sklearn.__version__)`. Below the editor, a terminal window is open, showing the command `python3 p26.py` and its output: `Scikit-learn Version: 1.5.1`. The terminal prompt is `saicharan@saish-MacBook-Air vscode_practice %`.

```
1 import sklearn  
2  
3 print("Scikit-learn Version:", sklearn.__version__)  
  
saicharan@saish-MacBook-Air vscode_practice % python3 p26.py  
Scikit-learn Version: 1.5.1  
saicharan@saish-MacBook-Air vscode_practice %
```

Result:

The experiment was executed successfully and the expected output was obtained.

Experiment 3

Aim:

To create and display a NumPy array containing the first 20 natural numbers.

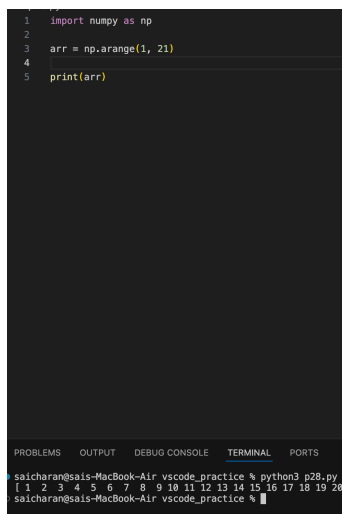
Algorithm:

1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
import numpy as np  
  
arr = np.arange(1, 21)  
  
print(arr)
```

Output:



The screenshot shows a VS Code editor with a Python script in a file named p28.py. The script imports numpy as np, creates an array arr using np.arange(1, 21), and prints it. The terminal at the bottom shows the command 'python3 p28.py' being executed, resulting in the output: [1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20].

```
1 import numpy as np  
2  
3 arr = np.arange(1, 21)  
4  
5 print(arr)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

saicharan@sais-MacBook-Air vscode_practice % python3 p28.py
[1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20]
saicharan@sais-MacBook-Air vscode_practice %

Result:

The experiment was executed successfully and the expected output was obtained.

Experiment 4

Aim:

To create and display a Pandas DataFrame using student names and grades.

Algorithm:

1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
import pandas as pd
```

```
data = {
```

```
    "Name": ["Sai", "Rahul", "Anu", "Priya"],
```

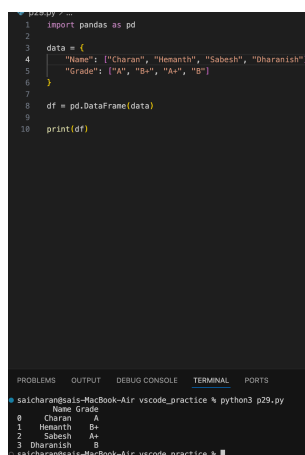
```
    "Grade": [90, 85, 95, 88]
```

```
}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

Output:



The screenshot shows a VS Code editor with a Python script in a file named `p29.py`. The script imports pandas as `pd`, creates a dictionary `data` with two keys: `"Name"` (values: `["Charan", "Hemanth", "Sabesh", "Dharanish"]`) and `"Grade"` (values: `["A", "B+", "A+", "B"]`), then creates a `DataFrame` from `data` and prints it. The terminal output shows the resulting DataFrame with columns `Name` and `Grade`, and rows indexed 0 to 3.

```
1 import pandas as pd
2
3 data = {
4     "Name": ["Charan", "Hemanth", "Sabesh", "Dharanish"],
5     "Grade": ["A", "B+", "A+", "B"]
6 }
7
8 df = pd.DataFrame(data)
9
10 print(df)
```

	Name	Grade
0	Charan	A
1	Hemanth	B+
2	Sabesh	A+
3	Dharanish	B

Result:

The experiment was executed successfully and the expected output was obtained.

Experiment 5

Aim:

To verify the successful installation of AI libraries by displaying their installed versions.

Algorithm:

1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
import numpy

import pandas

import matplotlib

import sklearn

import torch

import transformers

print("NumPy:", numpy.__version__)

print("Pandas:", pandas.__version__)

print("Matplotlib:", matplotlib.__version__)

print("Scikit-learn:", sklearn.__version__)

print("PyTorch:", torch.__version__)

print("Transformers:", transformers.__version__)
```

Output:



```
P 03/10/2024
1 import numpy
2 import pandas
3 import matplotlib
4 import sklearn
5
6 print("NumPy Version: ", numpy.__version__)
7 print("Pandas Version: ", pandas.__version__)
8 print("Matplotlib Version: ", matplotlib.__version__)
9 print("Scikit-learn Version: ", sklearn.__version__)
10
11 print("PyTorch Version: ", torch.__version__)
12 print("Transformers Version: ", transformers.__version__)

~/Desktop/output/03/10/2024/03/10/2024 - TERMINAL - 03/10/2024
$ cd ~/Desktop/output/03/10/2024/03/10/2024 & python p4b.py
NumPy Version: 1.26.4
Pandas Version: 2.2.2
Matplotlib Version: 3.8.4
Scikit-learn Version: 1.5.2
PyTorch Version: 2.3.0
Transformers Version: 4.41.0
```

Result:

The experiment was executed successfully and the expected output was obtained.

Experiment 6

Aim:

To create tensors using PyTorch and perform basic mathematical operations on them.

Algorithm:

1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
import torch

a = torch.tensor([1, 2, 3])

b = torch.tensor([4, 5, 6])

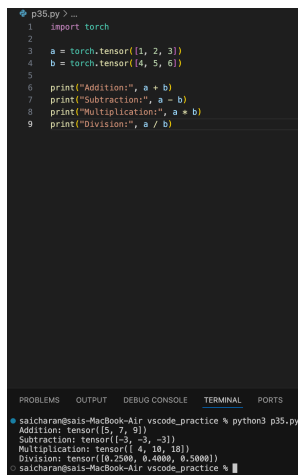
print("Addition:", a + b)

print("Subtraction:", a - b)

print("Multiplication:", a * b)

print("Division:", a / b)
```

Output:



```
p35.py? ...
1  import torch
2
3  a = torch.tensor([1, 2, 3])
4  b = torch.tensor([4, 5, 6])
5
6  print("Addition:", a + b)
7  print("Subtraction:", a - b)
8  print("Multiplication:", a * b)
9  print("Division:", a / b)

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
saicharan@saicharan-MacBook-Air vscode_practice % python3 p35.py
Addition: tensor([5, 7, 9])
Subtraction: tensor([-3, -3, -3])
Multiplication: tensor([4, 10, 18])
Division: tensor([0.2500, 0.4000, 0.5000])
saicharan@saicharan-MacBook-Air vscode_practice %
```

Result:

The experiment was executed successfully and the expected output was obtained.

Experiment 7

Aim:

To load a pre-trained transformer model using Hugging Face and perform sentiment analysis on text.

Algorithm:

1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

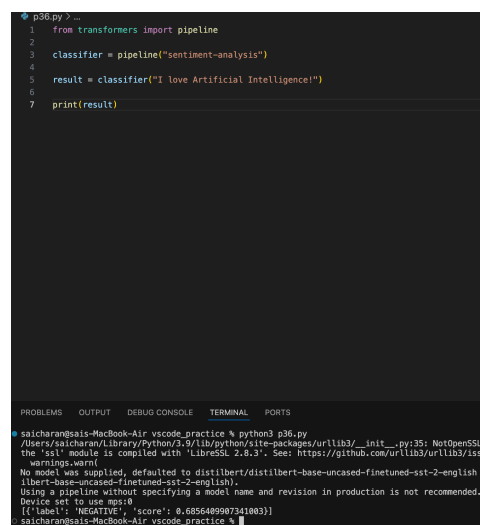
```
from transformers import pipeline
```

```
classifier = pipeline("sentiment-analysis")
```

```
result = classifier("I love Artificial Intelligence!")
```

```
print(result)
```

Output:



```
p36.py > ...
1 from transformers import pipeline
2
3 classifier = pipeline("sentiment-analysis")
4
5 result = classifier("I love Artificial Intelligence!")
6
7 print(result)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
saicharan@sais-MacBook-Air vscde_practice % python3 p36.py
/Users/saicharan/Library/Python/3.9/lib/python/site-packages/urllib3/_init_.py:35: NotOpenSSL
the 'ssl' module is compiled with 'LibreSSL 2.6.3'. See: https://github.com/urllib3/urllib3/iss
warnings.warn(
No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned-sst-2-english
libert-base-uncased-finetuned-sst-2-english).
Using a pipeline without specifying a model name and revision in production is not recommended.
Device set to use mps:0
[{'label': 'POSITIVE', 'score': 0.6856489907341003}]
saicharan@sais-MacBook-Air vscde_practice %
```

Result:

The experiment was executed successfully and the expected output was obtained.

Experiment 8

Aim:

To tokenize a sentence using the BERT tokenizer.

Algorithm:

1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
from transformers import BertTokenizer
```

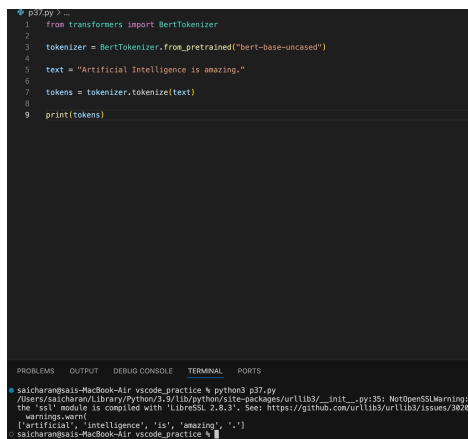
```
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
```

```
text = "Artificial Intelligence is amazing."
```

```
tokens = tokenizer.tokenize(text)
```

```
print(tokens)
```

Output:



```
p37.py > ...
1 from transformers import BertTokenizer
2
3 tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
4
5 text = "Artificial Intelligence is amazing."
6
7 tokens = tokenizer.tokenize(text)
8
9 print(tokens)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
saicharan@saicharan-MacBook-Air: ~ % python3 p37.py
/Users/saicharan/Library/Python/3.9/lib/python/site-packages/urllib3/_init_.py:35: NotOpenSSLWarning:
the 'ssl' module is compiled with 'libressl 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3000
warnings.warn(
[artificial, intelligence, is, amazing, .])
saicharan@saicharan-MacBook-Air: ~ %
```

Result:

The experiment was executed successfully and the expected output was obtained.

Experiment 9

Aim:

To generate text from a given prompt using a pre-trained GPT-2 language model.

Algorithm:

1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

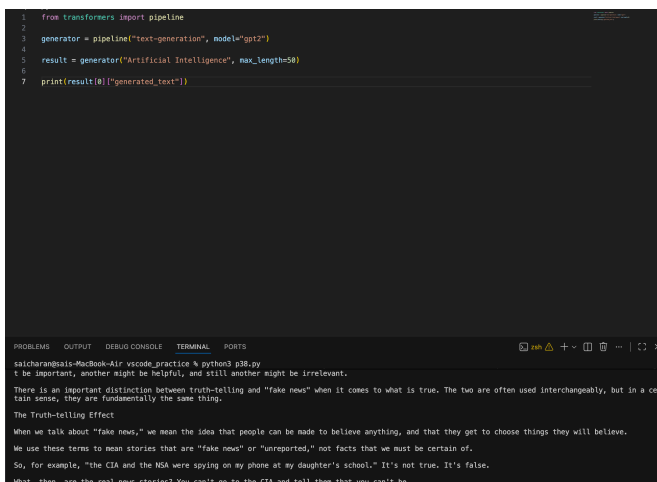
```
from transformers import pipeline
```

```
generator = pipeline("text-generation", model="gpt2")
```

```
result = generator("Artificial Intelligence", max_length=50)
```

```
print(result[0]["generated_text"])
```

Output:



```
1 from transformers import pipeline
2
3 generator = pipeline("text-generation", model="gpt2")
4
5 result = generator("Artificial Intelligence", max_length=50)
6
7 print(result[0]["generated_text"])
```

talcharan@sis-MacBook-Air vscode.practice % python3 g38.py

It be important, another might be helpful, and still another might be irrelevant.

There is an important distinction between truth-telling and "fake news" when it comes to what is true. The two are often used interchangeably, but in a certain sense, they are fundamentally the same thing.

The Truth-telling Effect

When we talk about "fake news," we mean the idea that people can be made to believe anything, and that they get to choose things they will believe.

We use these terms to mean stories that are "fake news" or "unreported," not facts that we must be certain of.

So, for example, "the CIA and the NSA were spying on my phone at my daughter's school." It's not true. It's false.

What, then, are the real news stories? You can't go to the CIA and tell them that you can't be

Result:

The experiment was executed successfully and the expected output was obtained.

Experiment 10

Aim:

To perform sentiment analysis using the Hugging Face pipeline() API with a pre-trained transformer model.

Algorithm:

1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

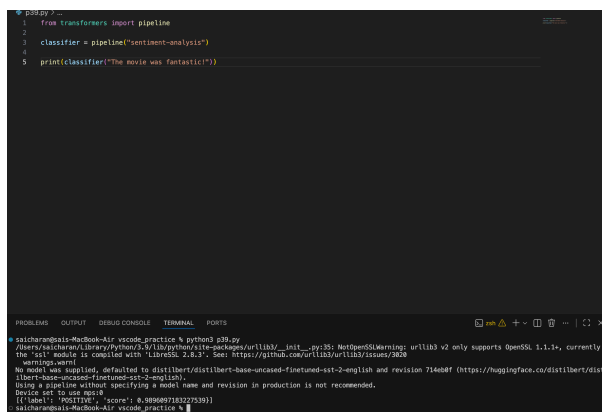
Code:

```
from transformers import pipeline
```

```
classifier = pipeline("sentiment-analysis")
```

```
print(classifier("The movie was fantastic!"))
```

Output:



```
python3 /...
1 from transformers import pipeline
2
3 classifier = pipeline("sentiment-analysis")
4
5 print(classifier("The movie was fantastic!"))

saicharan@saia-MacBook-Air: ~ % python3 p38.py
Warning:
We model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned-sts-b2-english and revision 714608f (https://huggingface.co/distilbert/distilbert-base-uncased-finetuned-sts-b2-english).
Using a pipeline without specifying a model name and revision in production is not recommended.
Device set to use MPS
[{"label": "POSITIVE", "score": 0.9896897183227535}]
saicharan@saia-MacBook-Air: ~ %
```

Result:

The experiment was executed successfully and the expected output was obtained.

Experiment 11

Aim:

To load the BERT tokenizer and tokenize a given sentence.

Algorithm:

1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
from transformers import BertTokenizer

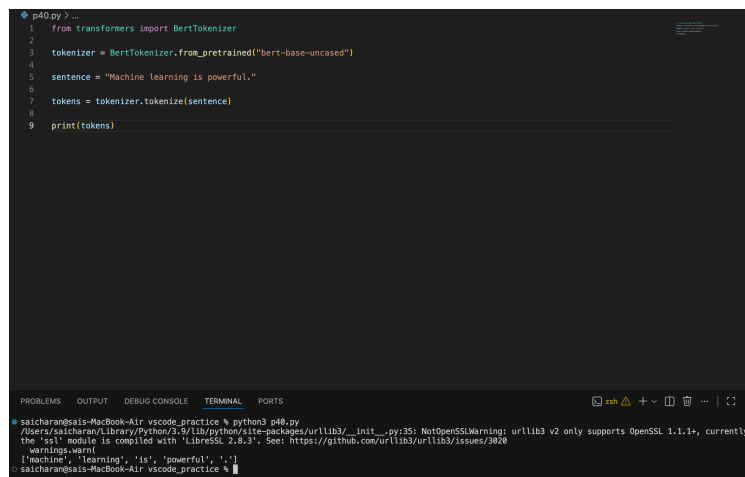
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

sentence = "Machine learning is powerful."

tokens = tokenizer.tokenize(sentence)

print(tokens)
```

Output:

A screenshot of a VS Code editor window. The editor shows a Python file named 'p40.py' with the following code:

```
1 from transformers import BertTokenizer
2
3 tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
4
5 sentence = "Machine learning is powerful."
6
7 tokens = tokenizer.tokenize(sentence)
8
9 print(tokens)
```

The terminal at the bottom shows the command being executed: `saicharan@sais-MacBook-Air vscode_practice % python3 p40.py`. The output is: `['machine', 'learning', 'is', 'powerful', '.']`. There is also a warning message about OpenSSL in the terminal output.

```
saicharan@sais-MacBook-Air vscode_practice % python3 p40.py
/Users/saicharan/Library/Python/2.5/lib/python/site-packages/urllib3/_init_.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently
the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
warnings.warn()
['machine', 'learning', 'is', 'powerful', '.']
saicharan@sais-MacBook-Air vscode_practice %
```

Result:

The experiment was executed successfully and the expected output was obtained.

Experiment 12

Aim:

To generate contextual word embeddings using a pre-trained BERT model.

Algorithm:

1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
from transformers import BertTokenizer, BertModel

import torch

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

model = BertModel.from_pretrained("bert-base-uncased")

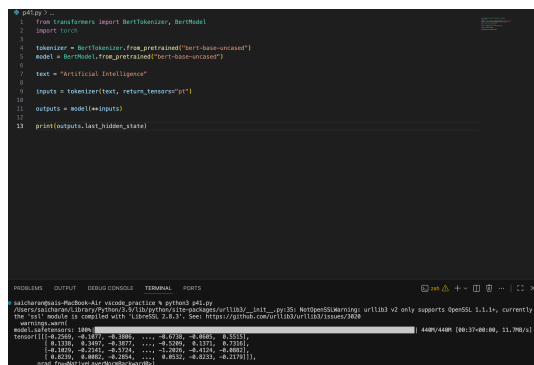
text = "Artificial Intelligence"

inputs = tokenizer(text, return_tensors="pt")

outputs = model(**inputs)

print(outputs.last_hidden_state)
```

Output:



```
from transformers import BertTokenizer, BertModel
import torch

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BertModel.from_pretrained("bert-base-uncased")

text = "Artificial Intelligence"

inputs = tokenizer(text, return_tensors="pt")

outputs = model(**inputs)

print(outputs.last_hidden_state)
```

4480/4480 (80.37-80.00, 11.796/s)

```
tensor([[[[ 2.0811,  0.1877, -0.1080, ..., -0.1728, -0.4626,  0.5513],
          [ 0.1130,  0.3407, -0.2072, ..., -0.5209,  0.5173,  0.7316],
          [ 0.1821, -0.2147, -0.2174, ..., -0.2851, -0.4214, -0.6881],
          [ 0.8239,  0.6802, -0.2054, ...,  0.8532, -0.8233, -0.2179]]]])
```

Result:

The experiment was executed successfully and the expected output was obtained.

Experiment 13

Aim:

To generate meaningful text using a pre-trained GPT-2 model.

Algorithm:

1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

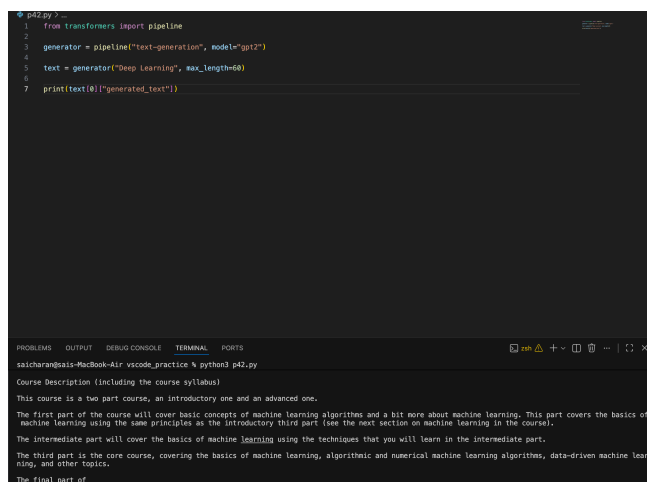
```
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

text = generator("Deep Learning", max_length=60)

print(text[0]["generated_text"])
```

Output:



The screenshot shows a VS Code editor with a Python file named p42.py. The code in the file is as follows:

```
1 from transformers import pipeline
2
3 generator = pipeline("text-generation", model="gpt2")
4
5 text = generator("Deep Learning", max_length=60)
6
7 print(text[0]["generated_text"])
```

Below the editor, the terminal window is open, showing the output of the script. The output is:

```
Course Description (including the course syllabus)
This course is a two part course, an introductory one and an advanced one.
The first part of the course will cover basic concepts of machine learning algorithms and a bit more about machine learning. This part covers the basics of
machine learning using the same principles as the introductory third part (see the next section on machine learning in the course).
The intermediate part will cover the basics of machine learning using the techniques that you will learn in the intermediate part.
The third part is the core course, covering the basics of machine learning, algorithmic and numerical machine learning algorithms, data-driven machine lear
ning, and other topics.
The final part of
```

Result:

The experiment was executed successfully and the expected output was obtained.

Experiment 14

Aim:

To perform question answering using a pre-trained transformer model.

Algorithm:

1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
from transformers import pipeline
```

```
qa = pipeline("question-answering")
```

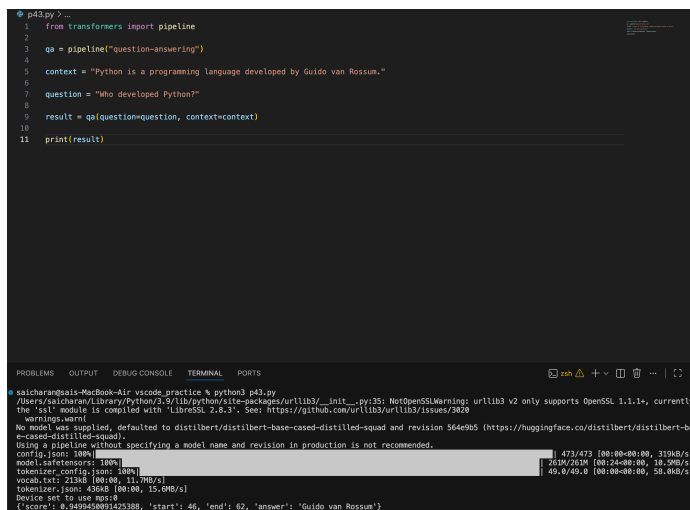
```
context = "Python is a programming language developed by Guido van Rossum."
```

```
question = "Who developed Python?"
```

```
result = qa(question=question, context=context)
```

```
print(result)
```

Output:



```
p43.py > ...
1 from transformers import pipeline
2
3 qa = pipeline("question-answering")
4
5 context = "Python is a programming language developed by Guido van Rossum."
6
7 question = "Who developed Python?"
8
9 result = qa(question=question, context=context)
10
11 print(result)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

saicharan@saicharan-MacBook-Air: % python3 p43.py

/Users/saicharan/Library/Python/3.9/lib/python/site-packages/urllib3/_init_.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3000

warnings.warn()

No model was supplied, defaulted to distilbert/distilbert-base-cased-distilled-squad and revision 564e905 (https://huggingface.co/distilbert/distilbert-base-cased-distilled-squad)

Using a pipeline without specifying a model name and revision in production is not recommended.

config.json: 100% | 471/473 [00:00:00, 319kB/s]

model.safetensors: 100% | 761M/761M [00:24:00:00, 18.3MB/s]

tokenizer_config.json: 100% | 49.8/49.8 [00:00:00:00, 58.8kB/s]

vocab.txt: 213kB [00:00, 11.7MB/s]

tokenizer.json: 426kB [00:00, 15.0MB/s]

Device set to use mps

{'score': 8.9499540991429108, 'start': 46, 'end': 62, 'answer': 'Guido van Rossum'}

Result:

The experiment was executed successfully and the expected output was obtained.

Experiment 15

Aim:

To tokenize a sentence using the BERT tokenizer and display the generated tokens along with their token IDs.

Algorithm:

1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

sentence = "Artificial Intelligence"

tokens = tokenizer.tokenize(sentence)

ids = tokenizer.convert_tokens_to_ids(tokens)

print("Tokens:", tokens)

print("Token IDs:", ids)
```

Output:

```
1 from transformers import BertTokenizer
2
3 tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
4
5 sentence = "Artificial Intelligence"
6
7 tokens = tokenizer.tokenize(sentence)
8
9 ids = tokenizer.convert_tokens_to_ids(tokens)
10
11 print("Tokens:", tokens)
12 print("Token IDs:", ids)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
saicharan@saicharan-MacBook-Air: ~/vscode_practice % python3 p44.py
/Users/saicharan/Library/Python/3.9/lib/python/site-packages/urllib3/_init_.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
warnings.warn(
Tokens: ['Artificial', 'Intelligence']
Token IDs: [7976, 4454]
saicharan@saicharan-MacBook-Air: ~/vscode_practice %
```

Result:

The experiment was executed successfully and the expected output was obtained.

Experiment 16

Aim:

To compare the tokenization results produced by the BERT and GPT-2 tokenizers.

Algorithm:

1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
from transformers import BertTokenizer, GPT2Tokenizer

bert = BertTokenizer.from_pretrained("bert-base-uncased")

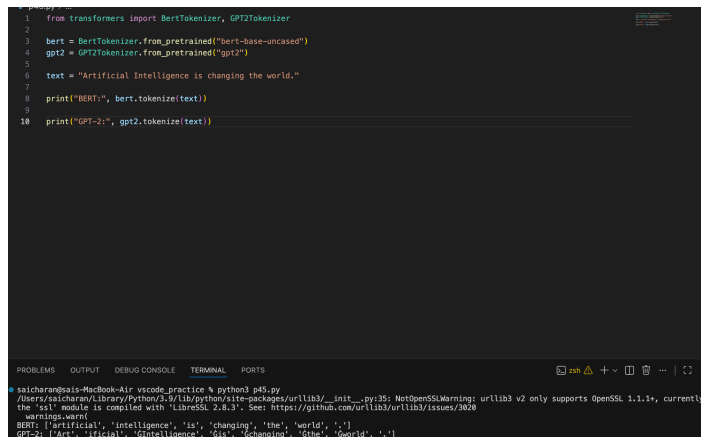
gpt2 = GPT2Tokenizer.from_pretrained("gpt2")

text = "Artificial Intelligence is changing the world."

print("BERT:", bert.tokenize(text))

print("GPT-2:", gpt2.tokenize(text))
```

Output:



```
1 from transformers import BertTokenizer, GPT2Tokenizer
2
3 bert = BertTokenizer.from_pretrained("bert-base-uncased")
4 gpt2 = GPT2Tokenizer.from_pretrained("gpt2")
5
6 text = "Artificial Intelligence is changing the world."
7
8 print("BERT:", bert.tokenize(text))
9
10 print("GPT-2:", gpt2.tokenize(text))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
saicharan@saicharan-MacBook-Air: ~/Library/Python/2.7/lib/python/site-packages/urllib3/_init_.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently
the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
warnings.warn(
BERT: ['Artificial', 'Intelligence', 'is', 'changing', 'the', 'world', '.']
GPT-2: ['<|endoftext|', 'Art', 'ificial', 'Intelligence', 'is', 'changing', 'the', 'world', '.']
```

Result:

The experiment was executed successfully and the expected output was obtained.

Experiment 17

Aim:

To generate contextual embeddings for a sentence using a pre-trained BERT model.

Algorithm:

1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
from transformers import BertTokenizer, BertModel

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

model = BertModel.from_pretrained("bert-base-uncased")

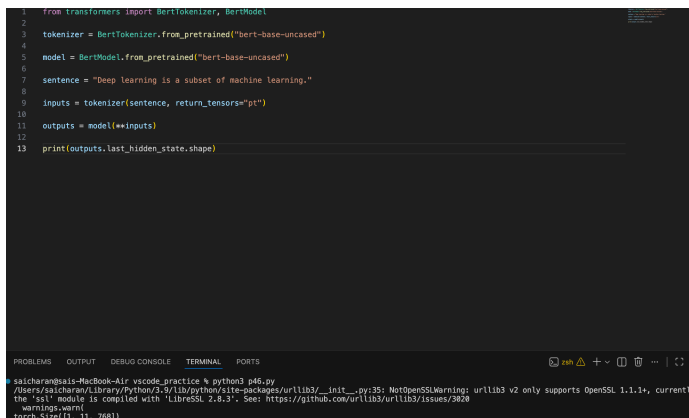
sentence = "Deep learning is a subset of machine learning."

inputs = tokenizer(sentence, return_tensors="pt")

outputs = model(**inputs)

print(outputs.last_hidden_state.shape)
```

Output:



```
1 from transformers import BertTokenizer, BertModel
2
3 tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
4
5 model = BertModel.from_pretrained("bert-base-uncased")
6
7 sentence = "Deep learning is a subset of machine learning."
8
9 inputs = tokenizer(sentence, return_tensors="pt")
10
11 outputs = model(**inputs)
12
13 print(outputs.last_hidden_state.shape)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

saicharan@saicharan-MacBook-Air: ~ % python3 p46.py

/Users/saicharan/Library/Python/3.9/lib/python/site-packages/urllib3/_init_.py:35: NetOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/2829

warnings.warn()

torch.Size([1, 1, 768])

Result:

The experiment was executed successfully and the expected output was obtained.

Experiment 18

Aim:

To compare the contextual embeddings generated by BERT for two semantically similar sentences.

Algorithm:

1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.

Code:

```
from transformers import BertTokenizer, BertModel

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

model = BertModel.from_pretrained("bert-base-uncased")

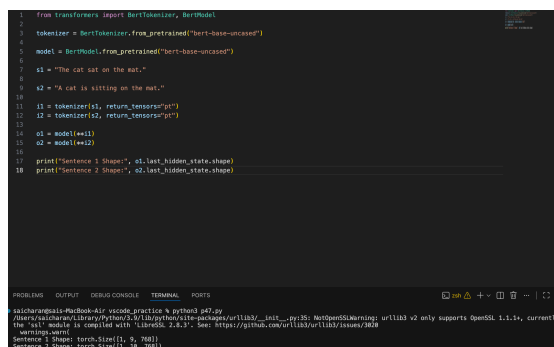
s1 = "The cat sat on the mat."
s2 = "A cat is sitting on the mat."

i1 = tokenizer(s1, return_tensors="pt")
i2 = tokenizer(s2, return_tensors="pt")

o1 = model(**i1)
o2 = model(**i2)

print("Sentence 1 Shape:", o1.last_hidden_state.shape)
print("Sentence 2 Shape:", o2.last_hidden_state.shape)
```

Output:



```
1 from transformers import BertTokenizer, BertModel
2
3 tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
4
5 model = BertModel.from_pretrained("bert-base-uncased")
6
7 s1 = "The cat sat on the mat."
8
9 s2 = "A cat is sitting on the mat."
10
11 i1 = tokenizer(s1, return_tensors="pt")
12 i2 = tokenizer(s2, return_tensors="pt")
13
14 o1 = model(**i1)
15 o2 = model(**i2)
16
17 print("Sentence 1 Shape:", o1.last_hidden_state.shape)
18 print("Sentence 2 Shape:", o2.last_hidden_state.shape)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

salcharan@salcharan-MacBook-Air: ~/code/practice % python3 p47.py
Name: salcharan@salcharan: /usr/bin/python3.5.10lib/python3.5/site-packages/urllib3/_init...py35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/2828
Sentence 1 Shape: torch.Size([1, 10])
Sentence 2 Shape: torch.Size([1, 10])

Result:

The experiment was executed successfully and the expected output was obtained.

Experiment 19

Aim:

To generate coherent text from a user-provided prompt using the GPT-2 language model.

Algorithm:

1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
from transformers import pipeline
```

```
generator = pipeline("text-generation", model="gpt2")
```

```
prompt = "The future of AI"
```

```
output = generator(prompt, max_length=80)
```

```
print(output[0]["generated_text"])
```

Output:

```
p48.py > -
1 from transformers import pipeline
2
3 generator = pipeline("text-generation", model="gpt2")
4
5 prompt = "The future of AI"
6
7 output = generator(prompt, max_length=80)
8
9 print(output[0]["generated_text"])
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

saicharan@sis-MacBook-Air: % python3 p48.py

Navigate to Command (⌘T) because of the way AI is changing the way people interact with information.

Scroll to Previous Command (⇧⌘T)

Scroll to Next Command (⇧⌘N)

The problem is not solved with the use of AI or the use of algorithms.

A lot of people are thinking of AI as the future of technology. But it turns out that we didn't even think of it that way before; we thought of it as the future of human action.

A lot of people are thinking about AI as the future of technology. But it turns out that we didn't even think of it that way before; we thought of it as the future of human action.

A lot of people are thinking of AI as the future of technology. But it turns out that we didn't even think of it that way before; we thought of it as the future of human action.

Result:

The experiment was executed successfully and the expected output was obtained.

Experiment 20

Aim:

To generate and compare responses for multiple prompts using a pre-trained Large Language Model (LLM).

Algorithm:

1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompts = [

    "Artificial Intelligence",

    "Machine Learning",

    "Future Technology"

]

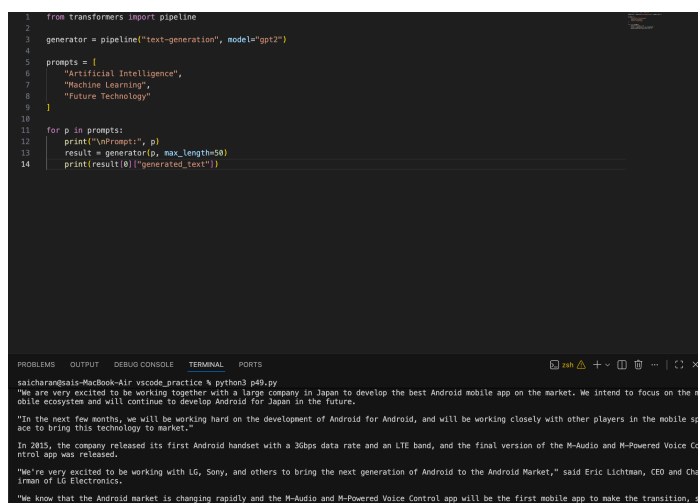
for p in prompts:

    print("\nPrompt:", p)

    result = generator(p, max_length=50)

    print(result[0]["generated_text"])
```

Output:



```
1 from transformers import pipeline
2
3 generator = pipeline("text-generation", model="gpt2")
4
5 prompts = [
6     "Artificial Intelligence",
7     "Machine Learning",
8     "Future Technology"
9 ]
10
11 for p in prompts:
12     print("\nPrompt:", p)
13     result = generator(p, max_length=50)
14     print(result[0]["generated_text"])
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

saicharan@saais-MacBook-Air: % python3 p49.py

"We are very excited to be working together with a large company in Japan to develop the best Android mobile app on the market. We intend to focus on the mobile ecosystem and will continue to develop Android for Japan in the future.

"In the next few months, we will be working hard on the development of Android for Android, and will be working closely with other players in the mobile space to bring this technology to market."

In 2015, the company released its first Android handset with a 3Gbps data rate and an LTE band, and the final version of the M-Audio and M-Powered Voice Control app was released.

"We're very excited to be working with LG, Sony, and others to bring the next generation of Android to the Android Market," said Eric Lichtman, CEO and Chairman of LG Electronics.

"We know that the Android market is changing rapidly and the M-Audio and M-Powered Voice Control app will be the first mobile app to make the transition, so we are excited to work with the LG

Result:

The experiment was executed successfully and the expected output was obtained.